Listing 36.7 Creating an integration test with `TestServer`

```
public class StatusMiddlewareTests
{
    [Fact]
    public async Task StatusMiddlewareReturnsPong()
    {
        var hostBuilder = new HostBuilder()                          ❶
            .ConfigureWebHost(webHost =>                             ❶
            {
                webHost.Configure(app =>                             ❷
                    app.UseMiddleware<StatusMiddleware>());          ❷
                webHost.UseTestServer();                             ❸
            });

        IHost host = await hostBuilder.StartAsync();                 ❹
        HttpClient client = host.GetTestClient();                    ❺

        var response = await client.GetAsync("/ping");               ❻

        response.EnsureSuccessStatusCode();                          ❼
        var content = await response.Content.ReadAsStringAsync();    ❽
        Assert.Equal("pong", content);                               ❽
    }
}
```

❶ Configures a HostBuilder to define the in-memory test app

❷ Adds the Status-Middleware as the only middleware in the pipeline

❸ Configures the host to use the TestServer instead of Kestrel

❹ Builds and starts the host

❺ Creates an HttpClient, or you can interact directly with the server object

❻ Makes an in-memory request, which is handled by the app as normal

❼ Verifies that the response was a success (2xx) status code

❽ Reads the body content and verifies that it contains “pong”

This test ensures that the test application defined by `HostBuilder` returns the expected value when it receives a request to the `/ping` path. The request is entirely in-memory, but from the point of view of